Studio Nx

JS 13 — initLoginForm()

2026-10-18

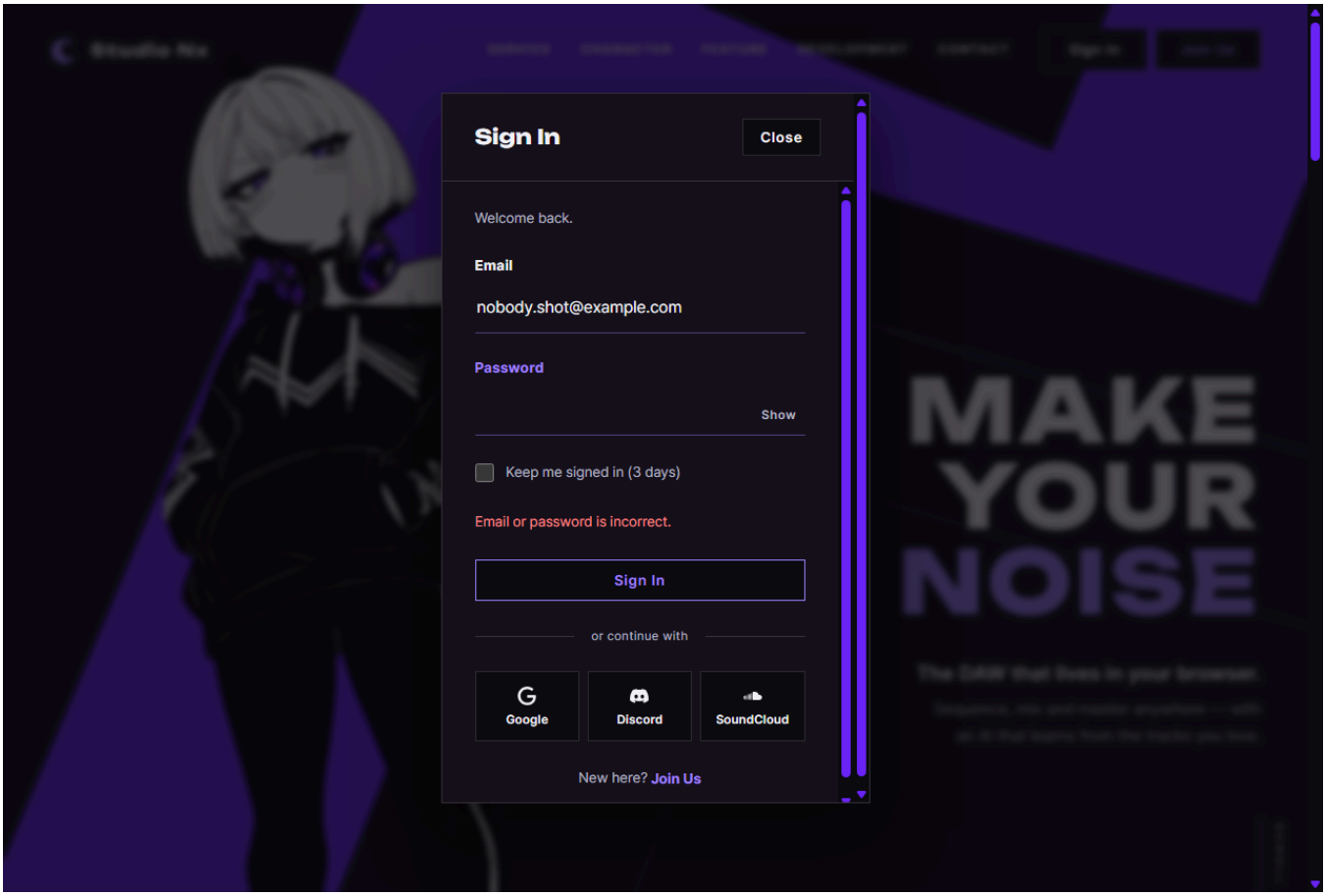
JS 13 — `initLoginForm()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-103, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	12 `initAuthState()` · 14 `initLoginModalFromHash()`

1. 하는 일

로그인 모달의 **Sign In** 버튼을 누르면 서버에 로그인하고, 결과에 따라 화면을 바꿉니다. **이 함수 덕분에 사람이 화면에서 직접 로그인할 수 있게 되었습니다.**

경우	화면
맞음, 주소에 `next` 없음 (home.html에서 Sign In)	모달이 닫히고 헤더가 **My Page, My Projects**로 바뀝니다
맞음, 주소에 `next` 있음 (`home.html#login?next=%2FmyPage.html`)	그 페이지(`/myPage.html`)로 이동합니다
틀림, 또는 없는 이메일	모달 안에 빨간 글씨로 "Email or password is incorrect."가 나오고, **비밀번호 칸만 비운 뒤** 그 칸에 커서를 둡니다
정지된 계정 (비밀번호는 맞음)	빨간 글씨로 "This account is disabled."가 나옵니다 (서버 문구)
빈 칸	브라우저가 "이 입력란을 작성하세요"라고 알리고, 서버로 보내지 않습니다



2. 언제 쓰나

로그인 모달이 있는 페이지가 열릴 때 한 번 부릅니다.

페이지	부르는 곳	성공하면
home.html	`home.js`의 `initLoginForm({ onSuccess: null })`	기본 동작을 합니다 (위의 표)
join.html	`join.js` (같은 방식)	기본 동작을 합니다. 17번 `initJoinForm`을 만들 때 `onSuccess`로 "홈으로 이동"을 넣을 예정입니다

누르면 일어나는 일

순서	일어나는 일	관련
①	빈 칸과 이메일 모양을 **브라우저가** 먼저 검사합니다 (HTML의 `required`, `type="email"`)	
②	Sign In 버튼을 비활성화하고(두 번 누르기 방지), 오류 글을 숨깁니다	
③	`POST /api/v1/auth/login {email, password, remember}`를 보냅니다	JS 02, 백엔드 32
④-성공	`setAccessToken(access_token)`으로 메모리에 저장합니다. 리프레시 쿠키는 서버가 심습니다	JS 01
⑤-성공	주소의 `next`를 확인해서 이동하거나, 모달을 닫고 헤더를 바꿉니다	JS 05 `parseLoginHash`
④-실패	오류 글을 보여 주고, 비밀번호 칸을 비운 뒤 커서를 둡니다	
⑥	버튼을 다시 켭니다	

3. 코드

import (파일 맨 위)

```
import { api, setAccessToken } from './api.js';
import { isLoggedIn, parseLoginHash } from './auth.js';
```

`api`, `setAccessToken`, `parseLoginHash`를 이번에 추가했습니다. 이 함수가 셋 다 씁니다 (`isLoggedIn`은 12번이 씁니다).

함수

```
export function initLoginForm(options = {}) {
  const form = document.getElementById('login-form');
  if (!form || form.dataset.loginReady === 'on') return;
  form.dataset.loginReady = 'on';

  const email = document.getElementById('login-email');
  const password = document.getElementById('login-password');
  const remember = document.getElementById('login-remember');
  const error = document.getElementById('login-error');
  const submit = form.querySelector('button[type="submit"]');

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    submit.disabled = true;
    try {
      const data = await api.post(
        '/auth/login',
        { email: email.value, password: password.value, remember: remember.checked },
        { auth: false },
      );
      setAccessToken(data.access_token);
      const next = parseLoginHash().next;
      if (options.onSuccess) {
        options.onSuccess(next);
      } else if (next) {
        location.href = next;
      } else {
        const modal = document.getElementById('login-modal');
        if (modal && modal.open) modal.close();
        document.documentElement.dataset.auth = 'in';
        form.reset();
      }
    } catch (err) {
      if (err.status === 401 || err.status === 422) {
        error.textContent = 'Email or password is incorrect.';
      } else {
        error.textContent = err.message || 'Something went wrong. Please try again.';
      }
      error.hidden = false;
      password.value = '';
      password.focus();
    } finally {
```

```
      submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export function initLoginForm({ onSuccess } = {}) {
  const form = document.getElementById('login-form');
  if (!form || form.dataset.loginReady) return;
  form.dataset.loginReady = 'on';
  const [email, password, remember, error] =
    ['login-email', 'login-password', 'login-remember', 'login-error'].map((id) => document.getElementBy
Id(id));
  const submit = form.querySelector('[type="submit"]');

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    submit.disabled = true;
    try {
      const { access_token } = await api.post('/auth/login',
        { email: email.value, password: password.value, remember: remember.checked }, { auth: false });
      setAccessToken(access_token);
      const { next } = parseLoginHash();
      if (onSuccess) return onSuccess(next);
      if (next) return location.assign(next);
      document.getElementById('login-modal')?.close();
      document.documentElement.dataset.auth = 'in';
      form.reset();
    } catch (err) {
      error.textContent = [401, 422].includes(err.status)
        ? 'Email or password is incorrect.'
        : err.message ?? 'Something went wrong. Please try again.';
      error.hidden = false;
      password.value = '';
      password.focus();
    } finally {
      submit.disabled = false;
    }
  });
}
```

최신 문법	실제 코드	뜻
`function f({ onSuccess } = {})`	`function f(options = {}) + options.onSuccess`	인자 객체에서 바로 꺼냅니다 (구조 분해)
`const [a, b] = [...].map(…)`	한 줄에 하나씩 `const a = …`	여러 변수를 한 번에 만듭니다
`const { access_token } = await …`	`const data = await …` + `data.access_token`	응답에서 칸을 꺼냅니다
`modal?.close()`	`if (modal && modal.open) modal.close()`	있을 때만 실행합니다

최신 문법	실제 코드	뜻
`[401, 422].includes(err.status)`	`err.status === 401 || err.status === 422`	둘 중 하나인지 확인합니다
`err.message ?? '…'`	`err.message || '…'`	없으면 대신 쓸 글입니다 (빈 문자열일 때는 결과가 다릅니다. 실제 코드는 빈 문자열일 때도 대신 쓸 글을 씁니다)

문법 풀이

import 문법	뜻
`import { api, setAccessToken } from './api.js'`	같은 폴더의 `api.js`에서 두 이름을 가져옵니다

식	뜻
`form.addEventListener('submit', …)`	폼을 보낼 때(버튼 클릭, Enter) 실행합니다. 브라우저 검사(`required`)를 **통과해야만** 이 일이 일어납니다
`event.preventDefault()`	브라우저가 페이지를 새로 여는 원래의 전송을 막습니다. 대신 JS가 보냅니다
`try { … } catch (err) { … } finally { … }`	실패하면 `catch`를, 성공이든 실패든 마지막에 `finally`를 실행합니다
`err.status`	`ApiError`의 HTTP 상태 번호입니다
`{ auth: false }`	로그인 요청에는 토큰을 붙이지도 않고, 재발급하지도 않습니다
`modal.open`, `modal.close()`	`<dialog>`가 열려 있는지 확인하고, 닫습니다
`form.reset()`	폼의 입력을 처음 상태로 되돌립니다
`location.href = next`	그 주소로 이동합니다

4. 흐름

1. 폼이 없거나 이미 연결했으면 끝냅니다.
2. 폼을 보낼 때 오류 글을 숨기고 버튼을 끕니다.
3. 로그인을 요청합니다. 성공하면 토큰을 저장하고, `onSuccess`를 부르거나 `next`로 이동하거나, 모달을 닫고 헤더를 바꿉니다.
4. 실패하면 오류 글을 보여 주고, 비밀번호를 비우고, 커서를 둡니다.
5. 버튼을 다시 켭니다.

5. 왜 이렇게

선택	이유
401과 422는 같은 문구로 보여 줌	서버(29번)와 같은 원칙으로, 무엇이 틀렸는지 알려 주지 않습니다. 422(형식 틀림)도 화면에서는 같은 문구입니다
403과 그 밖의 오류는 서버 문구를 그대로 보여 줌	정지된 계정("This account is disabled.")은 비밀번호가 맞은 본인에게만 오는 응답이라서 그대로 보여 줍니다. 네트워크 오류도 `api.js`의 문구로 보여 줍니다
실패하면 **비밀번호만** 비움	대부분 비밀번호를 잘못 친 경우입니다. 이메일은 다시 치지 않아도 되게 남겨 둡니다
전송 중에는 버튼을 끔	두 번 눌러서 요청이 두 번 가는 것을 막습니다
`onSuccess`가 없으면 기본 동작 (템플릿은 페이지마다 넘기라고 되어 있었음)	home.html의 `onSuccess`가 `null`이어도 바로 쓸 수 있습니다. 다른 동작이 필요한 페이지(join.html)만 넘기면 됩니다

선택	이유
`next`는 `parseLoginHash()`로 얻음	JS 05가 `safeNextPath`로 검사한 값이라서, 바깥 사이트로 보내지는 일이 없습니다
헤더는 다시 확인하지 않고 `data-auth="in"`으로 바꿈	방금 로그인에 성공했으므로 `/auth/refresh`를 다시 부를 필요가 없습니다

6. 시험 (실제 home.html에서 실제 서버로, 12개)

시험 계정은 실제 가입 API로 만들었고, 끝난 뒤 DB에서 지웠습니다.

시험	확인한 것
연결	`home.js`가 페이지를 열 때 이미 연결했습니다
틀린 비밀번호	전송 중에 버튼이 꺼지고, "Email or password is incorrect."가 나오며, 이메일은 남고 비밀번호 칸은 비워지며 커서가 비밀번호 칸에 있습니다. 버튼은 다시 켜지고, 토큰은 없으며, 모달은 열려 있습니다
없는 이메일	같은 문구가 나옵니다
빈 칸	`/auth/login` 요청이 하나도 나가지 않습니다 (브라우저 검사)
성공 (`next` 없음)	오류 글이 숨겨지고, 모달이 닫히며, `data-auth="in"`이 되고, 폼이 비워집니다. 메모리 토큰이 있고, 재발급도 200입니다 (쿠키도 생겼습니다)
성공 (`#login?next=%2FmyPage.html%3Ftab%3Dusage`)	`/myPage.html?tab=usage`로 이동하려고 합니다 (시험에서는 브라우저의 Navigation API로 이동만 막고 목적지를 읽었습니다)
`onSuccess`를 준 경우	`onSuccess('/myPage.html?tab=usage')`로 불립니다
두 번 부름	중복으로 연결되지 않습니다

12개 모두 통과했습니다. 위의 캡처는 없는 이메일과 틀린 비밀번호로 보낸 뒤에 찍었습니다.